

CIFAR100_Report

March 17, 2025

1 CIFAR-100 Classification Project Report

Author: Adam Szostek

1.1 1. Introduction

This report compares the performance of two models on the CIFAR-100 dataset: 1. Custom Convolutional Neural Network (CNN) 2. Fine-tuned ResNet-18

The project aims to evaluate which architecture performs better on this 100-class image classification task.

1.2 2. Environment Setup

```
[1]: import torch
import torch.nn as nn
import torchvision
from torchvision import transforms, datasets, models
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
from src.backbone_data_loader import testloader as resnet_testloader
import sys
import os
from PIL import Image
import json
import pickle
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

sys.path.append('src/')
from conv import Conv

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

torch.manual_seed(42)
```

```
np.random.seed(42)

plt.style.use('ggplot')
%matplotlib inline
```

Downloading <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz> to
./data/cifar-100-python.tar.gz

100%| | 169001437/169001437 [00:43<00:00, 3926561.07it/s]

Extracting ./data/cifar-100-python.tar.gz to ./data

Files already downloaded and verified

Using device: cuda

1.3 3. Data Loading

```
[2]: from src.backbone_data_loader import testloader as resnet_testloader
     from src.cifar_data_loader import c_testloader as test_loader

     # Class names
     with open('data/cifar-100-python/meta', 'rb') as f:
         meta = pickle.load(f)
     class_names = meta['fine_label_names']
```

Files already downloaded and verified

Files already downloaded and verified

1.4 4. Model Architectures

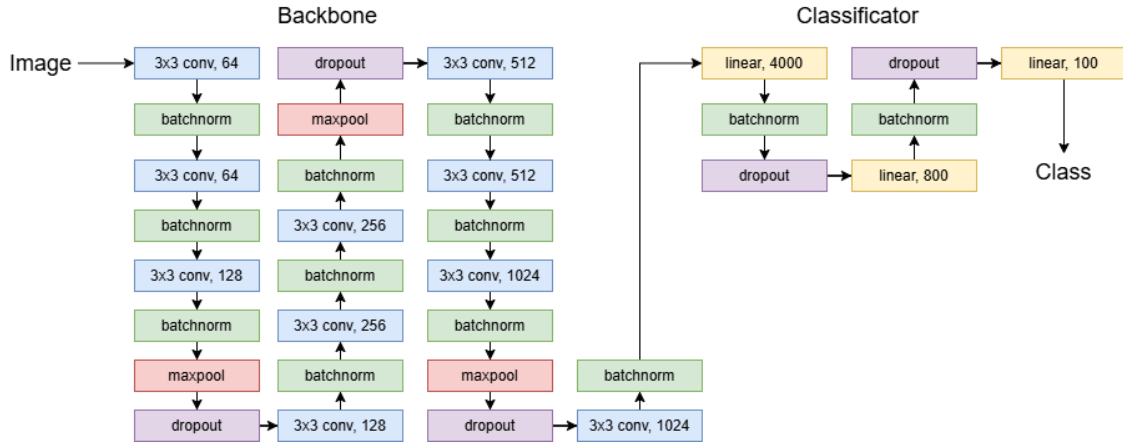
```
[3]: # Custom CNN
     custom_model = Conv(num_classes=100).to(device)

     # ResNet-18
     resnet18 = models.resnet18()
     num_fts = resnet18.fc.in_features
     resnet18.fc = nn.Sequential(nn.Linear(num_fts, 100))
     resnet18 = resnet18.to(device)
```

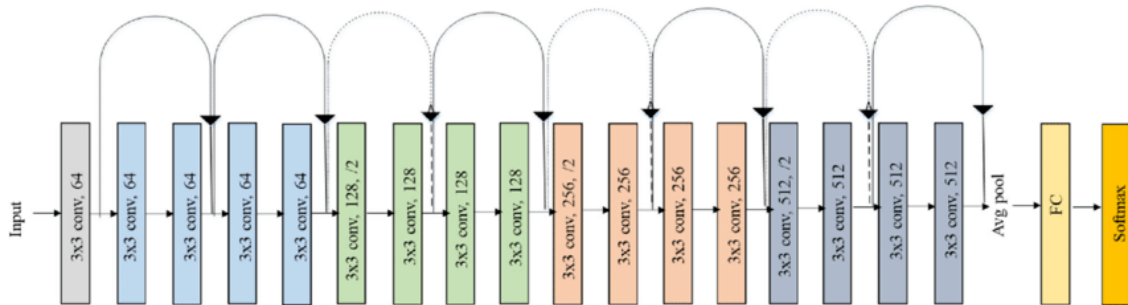
```
[4]: from IPython.display import Image, display

     print("Custom CNN architecture")
     display(Image(filename='models/Custom_CNN_architecture.drawio.png'))
     print("Resnet18 architecture")
     display(Image(filename='models/resnet18_architecture.png'))
```

Custom CNN architecture



Resnet18 architecture

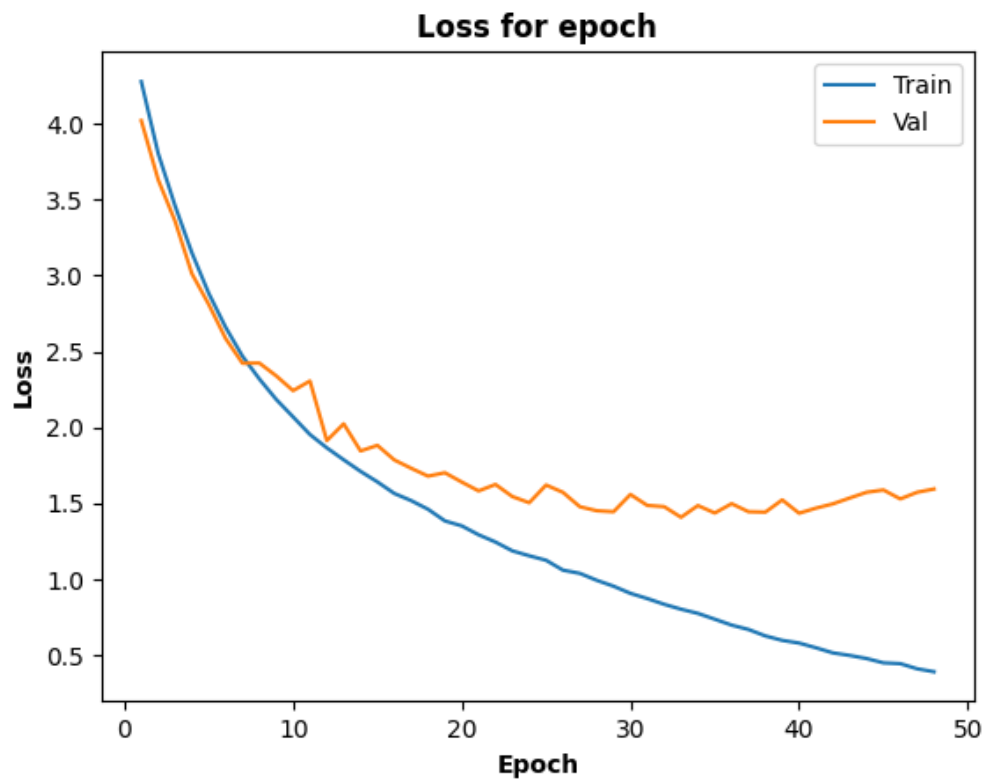


1.5 5. Model Training History Visualization

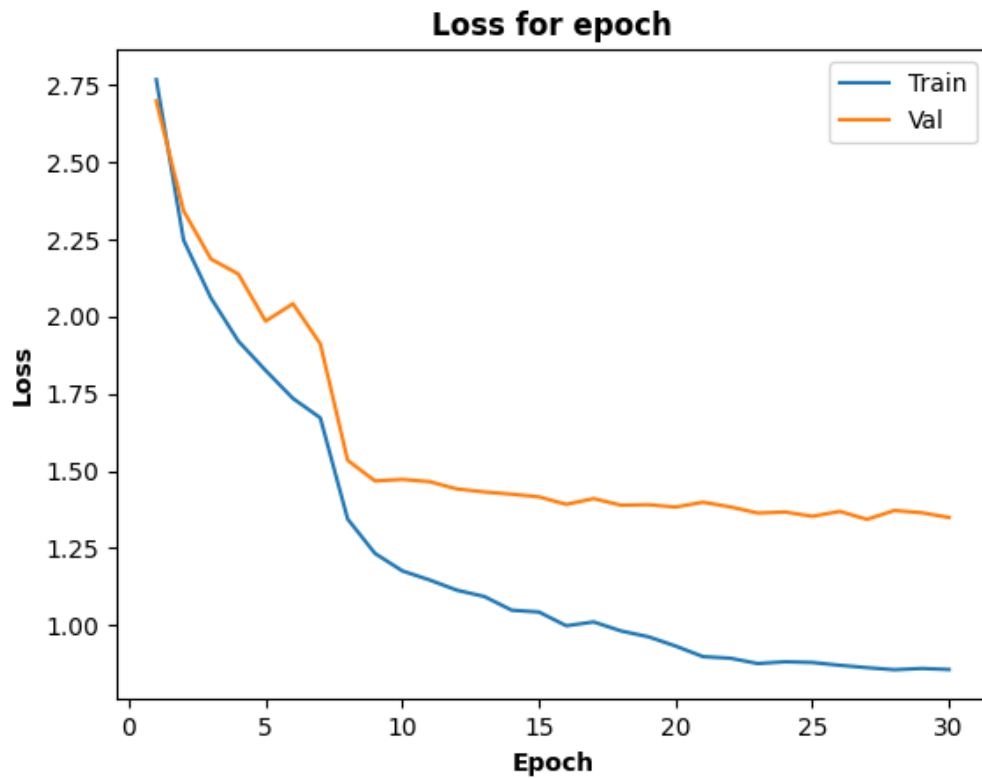
```
[6]: from IPython.display import Image, display

print("Custom CNN training")
display(Image(filename='results/conv_training.png'))
print("Resnet18 training")
display(Image(filename='results/resnet_training.png'))
```

Custom CNN training



Resnet18 training



1.6 6. Model Evaluation

```
[4]: def evaluate_model(model, model_path, test_loader):
    model.load_state_dict(torch.load(model_path))
    model.eval()

    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)
            labels = labels.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    return all_labels, all_preds
```

```

custom_labels, custom_preds = evaluate_model(custom_model, 'models/best_model.
    ↪pth', test_loader)
resnet_labels, resnet_preds = evaluate_model(resnet18, 'models/resnet18.pth',
    ↪resnet_testloader)

```

1.7 7. Performance Metrics

```

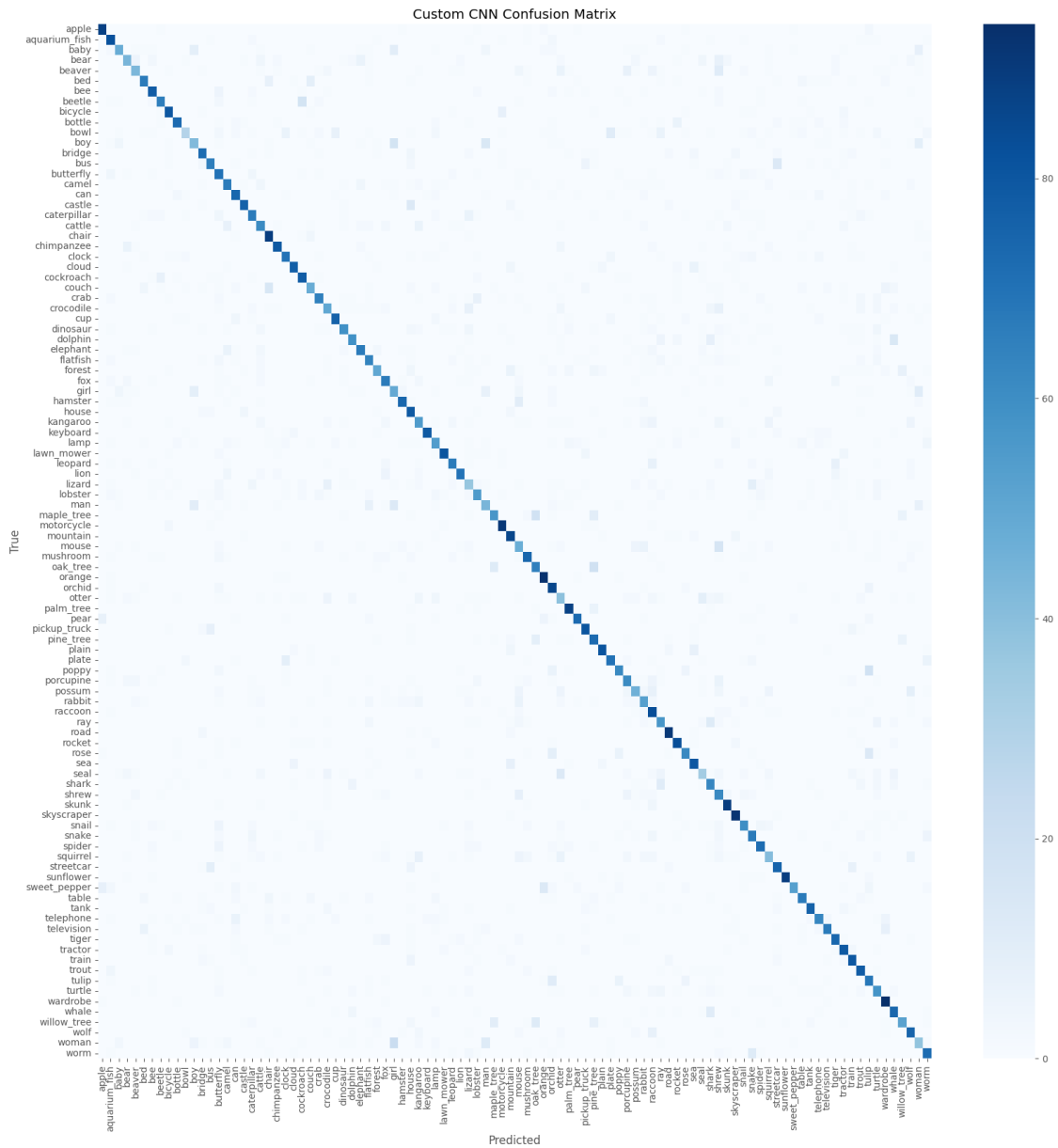
[5]: def plot_confusion_matrix(labels, preds, class_names, title):
    cm = confusion_matrix(labels, preds)
    plt.figure(figsize=(20, 20))
    sns.heatmap(cm, annot=False, fmt='d', cmap='Blues',
        xticklabels=class_names, yticklabels=class_names)
    plt.title(title)
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.show()

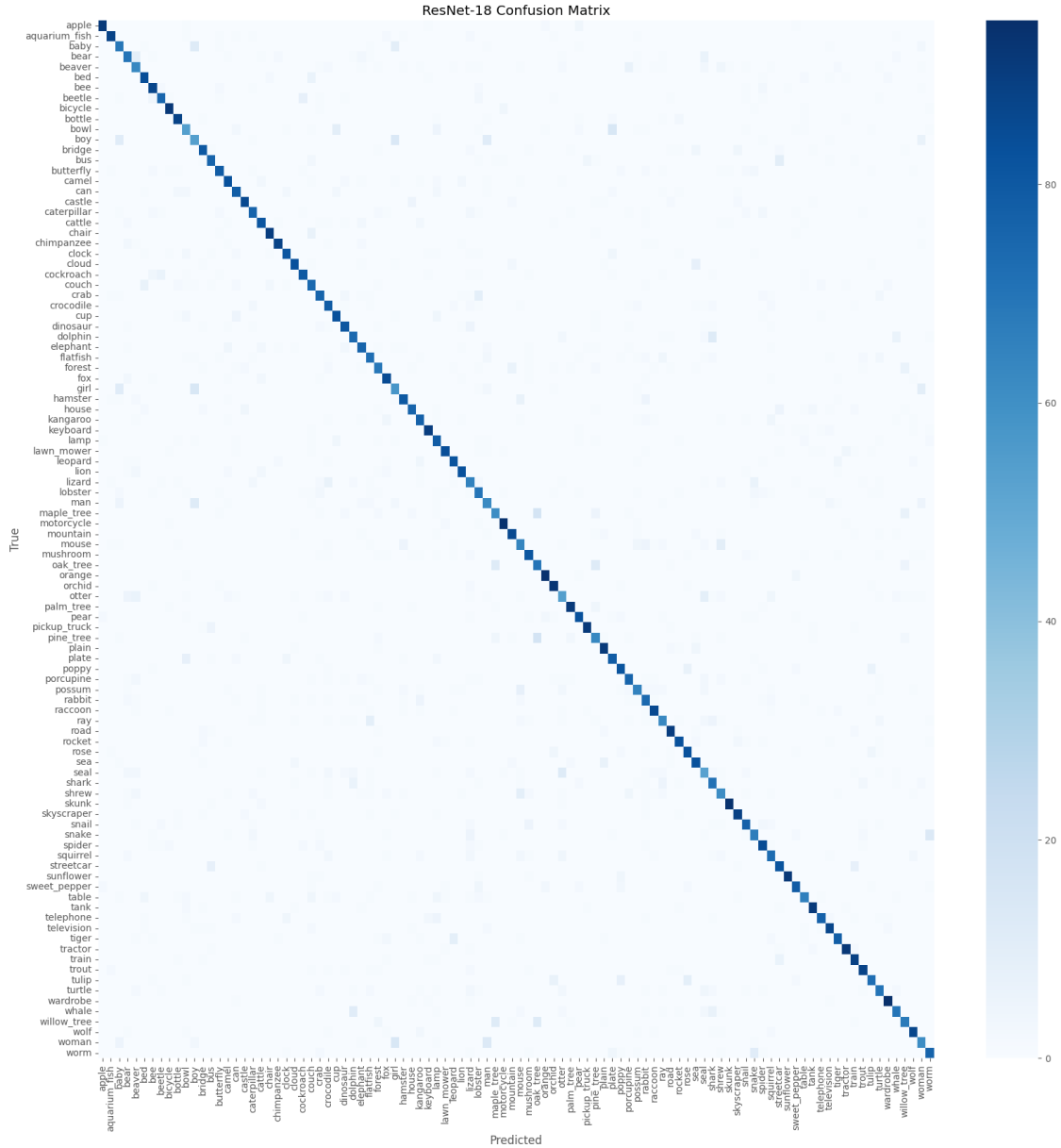
# Confusion matrices
plot_confusion_matrix(custom_labels, custom_preds, class_names, 'Custom CNN
    ↪Confusion Matrix')
plot_confusion_matrix(resnet_labels, resnet_preds, class_names, 'ResNet-18
    ↪Confusion Matrix')

# Classification reports
print("Custom CNN Classification Report:")
print(classification_report(custom_labels, custom_preds,
    ↪target_names=class_names, zero_division=0))

print("\nResNet-18 Classification Report:")
print(classification_report(resnet_labels, resnet_preds,
    ↪target_names=class_names, zero_division=0))

```





Custom CNN Classification Report:

	precision	recall	f1-score	support
apple	0.84	0.88	0.86	100
aquarium_fish	0.68	0.82	0.74	100
baby	0.73	0.48	0.58	100
bear	0.62	0.44	0.51	100
beaver	0.63	0.43	0.51	100
bed	0.77	0.67	0.72	100
bee	0.73	0.78	0.75	100

beetle	0.79	0.66	0.72	100
bicycle	0.79	0.80	0.80	100
bottle	0.81	0.73	0.77	100
bowl	0.67	0.29	0.41	100
boy	0.55	0.42	0.47	100
bridge	0.73	0.72	0.72	100
bus	0.71	0.66	0.68	100
butterfly	0.48	0.70	0.57	100
camel	0.68	0.67	0.67	100
can	0.68	0.74	0.71	100
castle	0.84	0.76	0.80	100
caterpillar	0.66	0.67	0.66	100
cattle	0.60	0.61	0.61	100
chair	0.71	0.89	0.79	100
chimpanzee	0.82	0.79	0.81	100
clock	0.70	0.70	0.70	100
cloud	0.82	0.77	0.79	100
cockroach	0.71	0.78	0.74	100
couch	0.62	0.48	0.54	100
crab	0.76	0.65	0.70	100
crocodile	0.54	0.50	0.52	100
cup	0.84	0.76	0.80	100
dinosaur	0.76	0.59	0.66	100
dolphin	0.71	0.59	0.64	100
elephant	0.78	0.67	0.72	100
flatfish	0.63	0.64	0.63	100
forest	0.68	0.50	0.58	100
fox	0.66	0.67	0.67	100
girl	0.42	0.49	0.45	100
hamster	0.84	0.74	0.79	100
house	0.54	0.79	0.64	100
kangaroo	0.55	0.55	0.55	100
keyboard	0.93	0.79	0.85	100
lamp	0.55	0.54	0.55	100
lawn_mower	0.89	0.81	0.85	100
leopard	0.74	0.67	0.70	100
lion	0.84	0.70	0.77	100
lizard	0.38	0.37	0.37	100
lobster	0.62	0.55	0.58	100
man	0.48	0.44	0.46	100
maple_tree	0.70	0.57	0.63	100
motorcycle	0.83	0.91	0.87	100
mountain	0.69	0.86	0.76	100
mouse	0.42	0.47	0.44	100
mushroom	0.76	0.74	0.75	100
oak_tree	0.61	0.65	0.63	100
orange	0.78	0.93	0.85	100
orchid	0.57	0.86	0.68	100

otter	0.38	0.39	0.39	100
palm_tree	0.80	0.88	0.84	100
pear	0.87	0.73	0.79	100
pickup_truck	0.76	0.81	0.79	100
pine_tree	0.54	0.73	0.62	100
plain	0.87	0.81	0.84	100
plate	0.75	0.70	0.73	100
poppy	0.74	0.62	0.67	100
porcupine	0.57	0.61	0.59	100
possum	0.61	0.46	0.52	100
rabbit	0.54	0.53	0.53	100
raccoon	0.50	0.83	0.62	100
ray	0.51	0.57	0.54	100
road	0.84	0.91	0.88	100
rocket	0.75	0.83	0.79	100
rose	0.73	0.62	0.67	100
sea	0.75	0.80	0.77	100
seal	0.49	0.34	0.40	100
shark	0.53	0.63	0.57	100
shrew	0.38	0.62	0.47	100
skunk	0.79	0.89	0.84	100
skyscraper	0.77	0.92	0.84	100
snail	0.67	0.60	0.63	100
snake	0.58	0.67	0.62	100
spider	0.73	0.72	0.72	100
squirrel	0.50	0.41	0.45	100
streetcar	0.68	0.75	0.71	100
sunflower	0.99	0.89	0.94	100
sweet_pepper	0.84	0.53	0.65	100
table	0.76	0.67	0.71	100
tank	0.84	0.76	0.80	100
telephone	0.75	0.63	0.68	100
television	0.79	0.67	0.72	100
tiger	0.73	0.74	0.74	100
tractor	0.67	0.79	0.72	100
train	0.69	0.80	0.74	100
trout	0.68	0.78	0.73	100
tulip	0.54	0.68	0.60	100
turtle	0.51	0.58	0.54	100
wardrobe	0.75	0.94	0.83	100
whale	0.65	0.74	0.69	100
willow_tree	0.70	0.52	0.60	100
wolf	0.64	0.75	0.69	100
woman	0.46	0.38	0.42	100
worm	0.67	0.72	0.69	100
accuracy			0.67	10000
macro avg	0.68	0.67	0.67	10000

weighted avg	0.68	0.67	0.67	10000
--------------	------	------	------	-------

ResNet-18 Classification Report:

	precision	recall	f1-score	support
apple	0.95	0.91	0.93	100
aquarium_fish	0.92	0.88	0.90	100
baby	0.62	0.69	0.65	100
bear	0.71	0.70	0.70	100
beaver	0.62	0.63	0.63	100
bed	0.82	0.84	0.83	100
bee	0.81	0.87	0.84	100
beetle	0.84	0.75	0.79	100
bicycle	0.91	0.92	0.92	100
bottle	0.86	0.88	0.87	100
bowl	0.62	0.54	0.58	100
boy	0.53	0.54	0.54	100
bridge	0.82	0.80	0.81	100
bus	0.78	0.76	0.77	100
butterfly	0.86	0.79	0.82	100
camel	0.81	0.83	0.82	100
can	0.76	0.78	0.77	100
castle	0.91	0.86	0.88	100
caterpillar	0.79	0.76	0.78	100
cattle	0.79	0.81	0.80	100
chair	0.90	0.90	0.90	100
chimpanzee	0.93	0.88	0.90	100
clock	0.85	0.81	0.83	100
cloud	0.87	0.83	0.85	100
cockroach	0.85	0.80	0.82	100
couch	0.67	0.74	0.70	100
crab	0.76	0.76	0.76	100
crocodile	0.69	0.79	0.73	100
cup	0.81	0.83	0.82	100
dinosaur	0.80	0.81	0.81	100
dolphin	0.74	0.74	0.74	100
elephant	0.77	0.76	0.76	100
flatfish	0.72	0.72	0.72	100
forest	0.71	0.69	0.70	100
fox	0.81	0.85	0.83	100
girl	0.54	0.58	0.56	100
hamster	0.85	0.80	0.82	100
house	0.85	0.76	0.80	100
kangaroo	0.74	0.77	0.75	100
keyboard	0.91	0.90	0.90	100
lamp	0.65	0.79	0.71	100
lawn_mower	0.92	0.84	0.88	100

leopard	0.80	0.82	0.81	100
lion	0.88	0.83	0.86	100
lizard	0.57	0.65	0.60	100
lobster	0.64	0.70	0.67	100
man	0.59	0.62	0.60	100
maple_tree	0.67	0.61	0.64	100
motorcycle	0.91	0.94	0.93	100
mountain	0.85	0.86	0.86	100
mouse	0.61	0.64	0.62	100
mushroom	0.81	0.81	0.81	100
oak_tree	0.60	0.70	0.65	100
orange	0.91	0.94	0.93	100
orchid	0.83	0.93	0.88	100
otter	0.57	0.54	0.56	100
palm_tree	0.90	0.91	0.91	100
pear	0.83	0.83	0.83	100
pickup_truck	0.93	0.91	0.92	100
pine_tree	0.66	0.63	0.64	100
plain	0.85	0.91	0.88	100
plate	0.72	0.79	0.76	100
poppy	0.77	0.82	0.79	100
porcupine	0.78	0.76	0.77	100
possum	0.75	0.65	0.70	100
rabbit	0.66	0.74	0.70	100
raccoon	0.85	0.86	0.86	100
ray	0.69	0.63	0.66	100
road	0.87	0.90	0.89	100
rocket	0.85	0.83	0.84	100
rose	0.81	0.83	0.82	100
sea	0.78	0.83	0.80	100
seal	0.55	0.53	0.54	100
shark	0.65	0.71	0.68	100
shrew	0.64	0.61	0.63	100
skunk	0.96	0.94	0.95	100
skyscraper	0.87	0.89	0.88	100
snail	0.79	0.77	0.78	100
snake	0.69	0.67	0.68	100
spider	0.78	0.85	0.81	100
squirrel	0.72	0.73	0.73	100
streetcar	0.75	0.82	0.78	100
sunflower	0.99	0.92	0.95	100
sweet_pepper	0.81	0.79	0.80	100
table	0.80	0.66	0.73	100
tank	0.89	0.91	0.90	100
telephone	0.90	0.77	0.83	100
television	0.84	0.87	0.86	100
tiger	0.85	0.78	0.81	100
tractor	0.91	0.92	0.92	100

train	0.80	0.89	0.84	100
trout	0.87	0.88	0.88	100
tulip	0.80	0.72	0.76	100
turtle	0.74	0.70	0.72	100
wardrobe	0.93	0.95	0.94	100
whale	0.80	0.70	0.74	100
willow_tree	0.74	0.68	0.71	100
wolf	0.87	0.87	0.87	100
woman	0.65	0.61	0.63	100
worm	0.78	0.76	0.77	100
accuracy			0.78	10000
macro avg	0.78	0.78	0.78	10000
weighted avg	0.78	0.78	0.78	10000

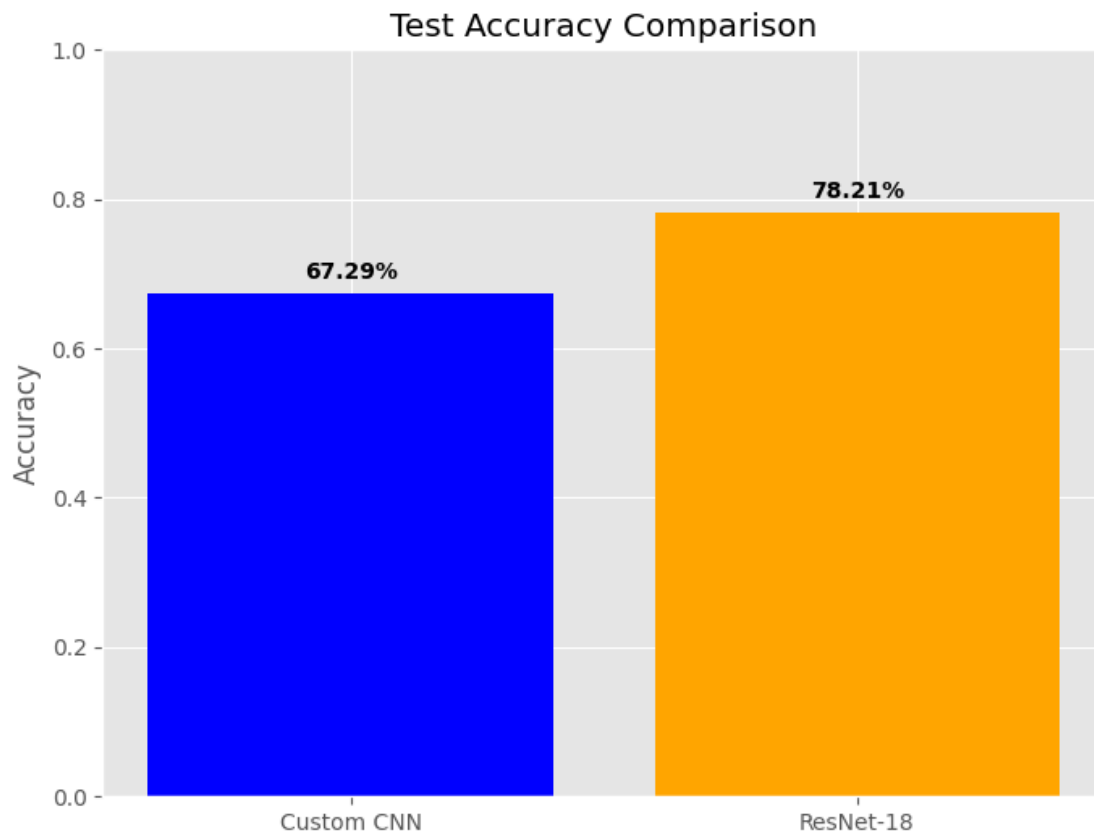
1.8 8. Accuracy Comparison

```
[6]: custom_accuracy = np.mean(np.array(custom_labels) == np.array(custom_preds))
resnet_accuracy = np.mean(np.array(resnet_labels) == np.array(resnet_preds))

plt.figure(figsize=(8, 6))
bars = plt.bar(['Custom CNN', 'ResNet-18'],
               [custom_accuracy, resnet_accuracy],
               color=['blue', 'orange'])
for bar in bars:
    plt.text(bar.get_x() + bar.get_width() / 2,
             bar.get_height() + 0.02,
             f'{bar.get_height():.2%}',
             ha='center',
             fontsize=10,
             fontweight='bold')

plt.title('Test Accuracy Comparison')
plt.ylabel('Accuracy')
plt.ylim(0, 1)

plt.show()
```

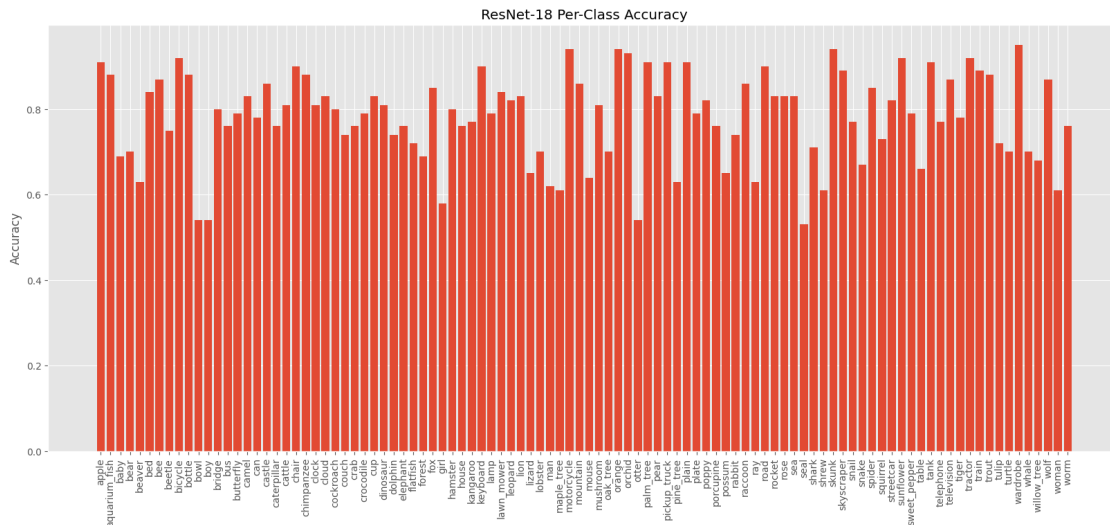
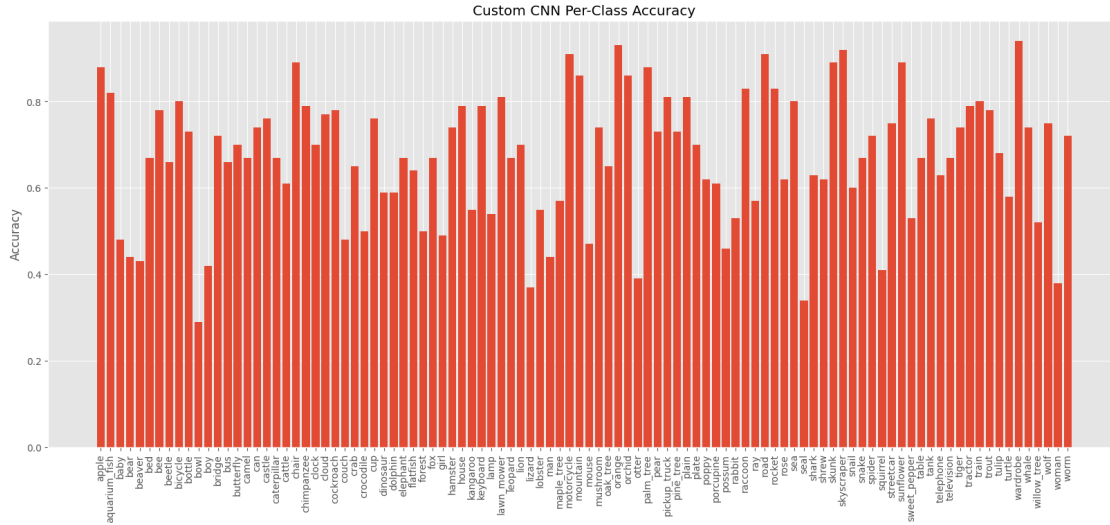


1.9 9. Per-Class Accuracy Analysis

```
[7]: def plot_class_accuracy(labels, preds, class_names, title):
    class_acc = []
    for i in range(100):
        class_mask = np.array(labels) == i
        class_acc.append(np.mean(np.array(preds)[class_mask] == i))

    plt.figure(figsize=(20, 8))
    plt.bar(range(100), class_acc)
    plt.xticks(range(100), class_names, rotation=90)
    plt.title(title)
    plt.ylabel('Accuracy')
    plt.show()

plot_class_accuracy(custom_labels, custom_preds, class_names, 'Custom CNN_
↳Per-Class Accuracy')
plot_class_accuracy(resnet_labels, resnet_preds, class_names, 'ResNet-18_
↳Per-Class Accuracy')
```



1.10 10. Sample Predictions

```
[8]: def show_sample_predictions(model, model_path, test_loader, class_names,
    ↪ num_samples=10):
    model.load_state_dict(torch.load(model_path))
    model.eval()

    dataiter = iter(test_loader)
    images, labels = next(dataiter)
    images = images.to(device)
    labels = labels.to(device)
```

```

outputs = model(images)
_, preds = torch.max(outputs, 1)

images = images.cpu().numpy()
labels = labels.cpu().numpy()
preds = preds.cpu().numpy()

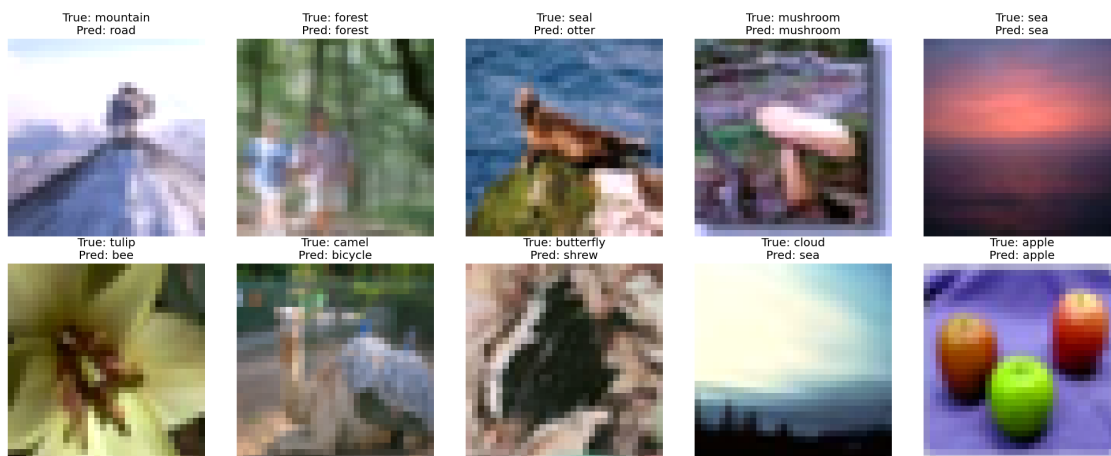
plt.figure(figsize=(20, 8))
for idx in range(num_samples):
    plt.subplot(2, 5, idx+1)
    img = np.transpose(images[idx], (1, 2, 0))
    img = img * np.array([0.2675, 0.2565, 0.2761]) + np.array([0.5071, 0.
↪4867, 0.4408])
    img = np.clip(img, 0, 1)
    plt.imshow(img)
    plt.title(f'True: {class_names[labels[idx]]}\nPred: ↪
↪{class_names[preds[idx]]}')
    plt.axis('off')
plt.tight_layout()
plt.show()

print("Custom CNN Sample Predictions:")
show_sample_predictions(custom_model, 'models/best_model.pth', test_loader, ↪
↪class_names)

print("\nResNet-18 Sample Predictions:")
show_sample_predictions(resnet18, 'models/resnet18.pth', resnet_testloader, ↪
↪class_names)

```

Custom CNN Sample Predictions:



ResNet-18 Sample Predictions:



1.11 11. Conclusion

1.11.1 Key Findings:

1. Accuracy Comparison:

- Custom CNN: 67.29%
- ResNet-18: 78.21%

2. Training Dynamics:

- Custom CNN
 - Loss Curves: The training loss decreases steadily over the epochs, indicating that the model is learning effectively. However, the gap between training and validation loss grows larger in later epochs, suggesting some degree of overfitting.
 - Stability: You see a bit more volatility in the validation curve, especially in the early epochs. This can be attributed to the network having fewer parameters (compared to ResNet-18) and being more sensitive to initialization or learning-rate changes.
 - Convergence: By around 20–30 epochs, the validation loss curve starts to plateau. Minor fluctuations thereafter reflect the network’s limited capacity to generalize further.
- ResNet-18
 - Loss Curves: The training loss also drops steadily, but the validation loss remains closer to the training curve than in the Custom CNN’s case—signaling better generalization.
 - Early Overfitting Check: You can see a brief phase of overfitting in the first 5–10 epochs, where training loss drops faster than validation loss. This is common for deep networks but is well-controlled, likely thanks to techniques such as weight decay, batch normalization, and skip connections inherent to ResNet.
 - Convergence: ResNet-18 converges more quickly and ends up at a lower overall loss, aligning with the higher final accuracy.

3. Class-wise Performance:

- High-Accuracy Classes: Certain classes (e.g., “apple,” “lion,” “tiger,” or others near the 80–

90% mark) are likely more visually distinctive, or they match well with the features learned by the networks.

- **Mid-Range Accuracy:** Some classes hover around 60–70%. These are often categories that share visual similarity with others (e.g., “bottle” vs. “bowl”) or are more variable in appearance (e.g., “cloud”).
- **Lower-Accuracy Classes:** A few classes dip closer to 50%. These tend to be inherently more ambiguous (e.g., “cockroach” vs. “beetle”) or might appear infrequently in the training set, leading to poorer generalization.

The ResNet-18 chart generally sits higher across most classes, reflecting its stronger representational capacity. The custom CNN’s class-wise pattern is similar (higher on distinctive classes, lower on visually confusing ones), but it lags behind ResNet-18 by ~10–15% on average.

4. Computational Efficiency:

- * Custom CNN: * Single epoch takes between 25 and 50 seconds depending on training devices (both cases use CUDA)
- * Pros: Faster per-batch training (smaller image size input), and lower memory usage.
- * Cons: The simpler architecture struggles to capture complex features, which limits peak accuracy.

- **ResNet-18:**
 - Single epoch takes between 1 minute 40 seconds and 3 minutes depending on training devices (similarly both cases use CUDA)
 - Pros: Deeper architecture with residual connections helps learn more complex patterns, leading to significantly higher accuracy (78.20% vs. 66.30%).
 - Cons: More computationally expensive. Although still relatively lightweight compared to larger ResNet variants, ResNet-18 requires more FLOPs and memory than a small custom CNN. In practice, if you have access to a GPU and can afford the extra training time, ResNet-18 is likely the better choice. If resources are very constrained, the custom CNN may be sufficient, though you sacrifice accuracy.

5. Summary:

- **Accuracy:** ResNet-18 outperforms the custom CNN (78.20% vs. 66.30%).
- **Generalization:** ResNet-18 shows a smaller gap between training and validation loss, indicating better generalization.
- **Class-wise:** Both networks do well on highly distinctive classes and less so on ambiguous or infrequent classes. ResNet-18’s deeper structure yields higher accuracy across the board.
- **Efficiency:** The custom CNN is more lightweight (smaller input size) but has a performance ceiling. ResNet-18 requires more compute but delivers higher accuracy.

Overall, the ResNet-18 model is the stronger performer for most applications, whereas the custom CNN may still be viable if computational resources or real-time constraints are strict.